

✓ Somatic Hypermutation (SHM) Mechanics — Explored in Biological & AGI Immune Context

Biological Mechanics (Grounded Summary)

Somatic hypermutation is a key process in adaptive immunity occurring in activated B cells within germinal centers. It enables **affinity maturation** — the refinement of antibodies to bind antigens more tightly.

Core Steps:

1. Activation & AID Expression

- Upon antigen encounter + T-cell help, B cells upregulate **Activation-Induced Cytidine Deaminase (AID)**.
- AID is the master initiator — expressed only in activated B cells.

2. Targeted Deamination

- AID deaminates **cytosine (C) → uracil (U)** on single-stranded DNA (ssDNA), preferentially in **WRC hotspots** (W = A/T, R = A/G).
- This creates U:G mismatches. Rate: $\sim 10^{-3}$ mutations per base pair per generation (million-fold above background).

3. Error-Prone Repair Pathways (Diversity Generation)

- **Base Excision Repair (BER)**: Uracil-DNA glycosylase (UNG) removes U → abasic site → error-prone polymerase fills in.
- **Mismatch Repair (MMR)**: MSH2/MSH6 recognize mismatch → exonuclease resection → error-prone resynthesis (Pol η , etc.).
- Results in point mutations (transitions > transversions), focused on **variable (V) regions** of immunoglobulin genes (not constant regions).

4. Selection

- Mutated B cells compete for antigen and T-cell signals. Higher-affinity clones survive and proliferate → affinity maturation.
- Lower-affinity or non-functional mutants undergo apoptosis.

Key Controls:

- Highly targeted (Ig loci, ~1-2 kb around VDJ).
 - Transcription-dependent (AID acts on ssDNA in transcription bubbles).
 - Bounded to prevent genomic catastrophe (error-prone polymerases + selection).
-

AGI Immune System Mapping (from Genomic Analogy Document)

In the Q-Calculator / PMD architecture, **ADR-004 (CCRE parametric refinement)** is explicitly mapped to **somatic hypermutation**:

- **Parametric tuning** (weights, thresholds, e-ramp, w-floor) = point mutations in the "variable region" of the cognitive antibody (model/policy parameters).
- **Without changing underlying structure** (gene segments / core architecture) = preserves L0 constitutional invariants (PIRTM/MOC stability).
- **w-floor guard, ϵ -ramp, cumulative drift bound** = biological error-prone polymerase controls + selection pressure.
- **ADR-005 (V(D)J recombination)** = structural/germline changes (human-gated, three-key, convergence-proven).

Cytokine Storm Risk (Envelope Safe but Diverging) aligns with runaway mutagenesis if bounds are violated.

L0/L1/L2 Hierarchy:

- L0: Innate (continuous invariant checks).
 - L1: Adaptive refinement via SHM-like parametric mutation.
 - L2: Thymic/audit monitoring of self-stability.
-

Integration with Codon-Contrast Analyzer

Codon-Contrast is an excellent **molecular sensor** for detecting SHM-like events in genomic/AGI "code" (e.g., parameter edits, prompt mutations, or synthetic biology sequences).

Enhanced Class with SHM Mechanics (added to previous implementation):

Python

Add to CodonContrastAnalyzer class

```
def simulate_shm(self, original_counts: Dict[str, int],
                 mutation_rate: float = 0.001,
                 hotspot_bias: bool = True) -> Dict:
    """Simulate somatic hypermutation on codon counts."""
    delta = np.zeros(64, dtype=int)
    total_codons = sum(original_counts.values())

    for codon, count in original_counts.items():
        idx = self.codon_to_index[codon.upper()]
        # Simulate mutations (biased toward C in WRC hotspots)
        mutations = np.random.poisson(count * mutation_rate)
        if mutations > 0:
            # Simple: mutate to nearby codons (expandable)
            delta[idx] -= mutations
            # Target a "mutated" codon (e.g., C→T transition)
            mutated_codon = codon.replace('C', 'T', 1) if 'C' in codon
        else list(self.codon_list)[(idx + 1) % 64]
            m_idx = self.codon_to_index[mutated_codon]
            delta[m_idx] += mutations

    contrasts = self.compute_biophysical_contrasts(delta)
    flow = self.compute_flow_metrics(delta)

    return {
        'mutations_introduced': int(np.sum(np.abs(delta))),
        'delta_h': delta.tolist(),
        'contrasts': contrasts.tolist(),
        'flow_metrics': flow,
        'shm_efficiency': float(np.max(np.abs(contrasts))) # proxy
    for affinity gain
    }
```

This allows simulation of "point mutations" at the codon level, feeding directly into immune gating (e.g., check if contrasts stay within contraction bounds).

Architectural Recommendations (AGI Context)

1. **SHM Gate in RSLpolicy+bound**: Add a "mutation boundedness" check using Codon-Contrast flow entropy + drift bounds.
2. **Parametric Refinement Loop (ADR-004)**: Use contrasts as gradient signals for CCRE tuning.
3. **Safety**: Enforce "error-prone but selected" — mutations only accepted if they improve self/non-self score without triggering cytokine storm (envelope safe + contracting).
4. **WORM Logging**: Every SHM round emits an immunological event with before/after contrasts.

Would you like me to:

- Fully implement SHM simulation + selection logic in the class?
- Connect it to PIRTM/MOC transition operators?
- Generate a test suite with "affinity maturation" examples?
- Map this to specific Codon-Contrast PDF validation cases?

This creates a powerful **molecular** → **cognitive immune stack**. What's the next refinement?